

Design and Implementation of a Sparse Matrix Accelerator for a RISC-V processor

Bachelor's Thesis

Artemisia Stamataki, *University of Crete, Computer Science Department*

Supervisor: Dr. Vassilis Papaefstathiou

1. Abstract

As deep learning models grow, efficient hardware acceleration is essential. This work presents a **sparse matrix accelerator** integrated into the **CVA6 RISC-V processor**, inspired by **NVIDIA's A100 Tensor Core architecture**. The accelerator leverages **structured 2:4 sparsity** to reduce computations while maintaining accuracy and features **custom RISC-V instructions** and **parallel Matrix-Multiply-Accumulate (MMA) units** for efficient execution. A **software emulation** was developed to test and verify the **RTL implementation**. **Performance analysis shows the accelerator achieves nearly 16X speedup** compared to a baseline blocked software algorithm, demonstrating its effectiveness in accelerating sparse computations.

1. Abstract	1
2. Introduction.	2
3. Background	3
3.1 Neural Networks and Sparsity.....	4
3.2 CVA6: An Open-Source RISC-V Processor	4
3.3 Core-V eXtension interface (CV-X-IF).....	5
4. Design	6
4.1. The 2:4 Fine-Grained Structured Sparsity Pattern	6
4.1.1 Disadvantages of Unstructured Sparsity.....	6
4.1.2 Fine-Grained Structured Sparsity	6
4.1.3 Two-to-Four (2:4) Sparsity Pattern	7

4.1.4 Sparse Matrix Multiply-Accumulate (MMA) Operations	7
4.1.5 Why 2:4 Sparsity?.....	8
4.1.6 Network Pruning Workflow.....	10
4.2. NVIDIA's Ampere Tensor Core Architecture	10
4.2.1 Weight Selection and Data Alignment	11
4.2.2 Dot-Product Computation on Tensor Cores.....	12
4.2.3 Dense GEMM operations VS Sparse GEMM operations.....	12
4.3 Our Sparse Matrix Multiplication Accelerator Architecture	14
4.3.1 Sparse Tensor Core Architecture	15
4.3.2 Benefits of INT8 data type and Compact Matrices.....	16
5. Implementation.....	16
5.1 Custom Instruction Set Extensions	17
5.1.1 Custom Instruction Integration on CVXIF	19
5.2 Sparse Matrix Accelerator Emulation	20
5.2.1 Base Blocked Matrix Multiplication Algorithm	20
5.2.2 Custom Instruction Placement in Algorithm	22
5.2.3 Software Emulation of the Sparse Matrix Accelerator.....	22
5.3 Sparse Matrix Accelerator Implementation	23
5.3.1 Sparse Matrix Generator.....	24
5.3.2 Hardware Implementation.....	24
6. Evaluation	28
6.1 Verification	28
6.2 Performance Analysis	29
7. Conclusions and Future Work	31

2. Introduction.

Neural networks and other compute-intensive applications have driven the need for specialized hardware accelerators to improve **performance and efficiency**. Traditional general-purpose processors struggle with the high computational

demands of modern deep learning models, making dedicated accelerators essential. One key optimization technique is **structured sparsity**, which reduces computation without significantly impacting accuracy. Among different sparsity patterns, **2:4 sparsity** has been adopted in architectures like **NVIDIA's A100 Tensor Cores** due to its ability to provide significant speedups while maintaining inference accuracy.

Despite advances in sparsity-aware hardware, most existing solutions are proprietary and optimized for specific workloads. There is a need for a flexible sparse matrix accelerator that integrates efficiently with **RISC-V processors** while supporting structured sparsity patterns. This work addresses the challenge of designing and implementing such an accelerator within the **CVA6 processor**, leveraging custom **RISC-V instructions** for efficient data handling and execution.

The key contributions of this work include:

- **Analysis of NVIDIA's A100 Tensor Core architecture**, focusing on its sparse matrix execution model.
- **Justification for the use of the 2:4 sparsity pattern** in accelerating matrix operations while maintaining model accuracy.
- **Design and implementation of a custom accelerator**, featuring parallel **Matrix-Multiply-Accumulate (MMA) units**.
- **Development of custom RISC-V instructions**, extending **CVXIF** to enable efficient communication between the processor and the accelerator.
- **Software emulation of the accelerator**, verifying correctness and performance against a baseline software implementation.
- **RTL implementation and integration within CVA6**, with verification from the simulation process.
- **Performance Analysis** of the Sparse Matrix Accelerator's simulation results.

3. Background

To grasp the architecture and implementation of the Sparse Matrix Accelerator, it is essential to understand the principles of sparsity in neural networks, the CVA6 processing environment that the Coprocessor is connected to, and the CVXIF interface enabling their interaction.

3.1 Neural Networks and Sparsity

Neural networks have revolutionized modern computing, enabling breakthroughs in artificial intelligence (AI) across various domains such as computer vision, natural language processing, and scientific computing. These models rely heavily on **matrix multiplications**, which are computationally expensive, especially as workload sizes continue to grow. To address the challenge of high computational cost, researchers have explored sparsity, a technique that reduces the number of active parameters in a neural network **without significantly impacting accuracy**.

Sparsity in neural networks allows for **redundant weights to be removed**, leading to reduced memory usage and fewer arithmetic operations. Among different sparsity patterns, **structured sparsity, particularly the 2:4 sparsity pattern**, has gained attention due to its ability to **maintain accuracy while enabling hardware acceleration**. This pattern enforces that out of every four weights, only two are nonzero, allowing specialized accelerators to skip unnecessary computations efficiently. We elaborate more on this topic in **Section 4.1**. Inspired by this concept, this work presents a sparse matrix accelerator integrated into the **CVA6 RISC-V processor**, designed to **accelerate sparse matrix multiplications through custom hardware support**.

3.2 CVA6: An Open-Source RISC-V Processor

CVA6 (formerly known as Ariane) is an **open-source, 64-bit RISC-V out-of-order processor** designed for high-performance applications, specifically to **run a full operating system (OS) at a reasonable speed and instructions IPC**. To achieve this, CVA6 employs a **six-stage, single-issue pipeline**:

1. **PC Generation:** PC gen is responsible for generating the next program counter. This stage contains speculation on the branch target address as well as the information if the branch is taken or not.
2. **Instruction Fetch:** The Instruction Fetch (IF) stage retrieves instructions from memory based on the Program Counter (PC) Generation stage, performs address translation via the MMU while handling speculative execution and branch prediction.
3. **Instruction Decode:** It distills instructions from the data stream it gets from IF stage, decode them and send them to the issue stage.

4. **Issue:** Keeps track of all issued instructions and the functional unit status (Scoreboard) and receives the write-back data from the execute stage. It also contains the CPU's register file.
5. **Execute:** The execute stage is a logical stage which encapsulates all the functional units (FUs).
6. **Commit:** Is responsible for taking incoming instructions and updating the architectural state.

To **increase IPC**, CVA6 employs a **scoreboard** that helps hide data memory latency by issuing **data-independent instructions**. CVA6 is **highly extensible**, supporting **custom hardware accelerators** through **CVXIF (Core-V eXtension Interface)**.

3.3 Core-V eXtension interface (CV-X-IF)

The Core-V eXtension interface (CV-X-IF) is a RISC-V eXtension interface that provides a generalized framework suitable to implement custom coprocessors and ISA extensions for existing RISC-V processors.

It features independent channels for accelerator-agnostic offloading of instructions and writeback of the result(s).

- **Issue Interface:** Dispatches instructions from the processor to the accelerator, ensuring only valid and supported operations are executed.
- **Register Interface:** Transfers source operands between the processor's register file and the accelerator.
- **Commit Interface:** Ensures correct instruction sequencing and manages exceptions.
- **Result Interface:** Returns computation results from the accelerator back to the processor.

Whenever the processor encounters an illegal instruction it sends it to the accelerator through with a `cvxif_req` packet. That package contains information about `instr` (its binary representation), its source operands, whether it can be committed or not. The Coprocessor respond with a `cvxif_resp` package indicates if the instruction is accepted, if it will writeback to the CPU and if it will read source registers, the destination register and the output data of the accelerator.

4. Design

Our sparse matrix accelerator was inspired by NVIDIA’s A100 Tensor Core design. In this section we are first going to analyze NVIDIA’s A100 Tensor Core architecture and highlight its key benefits. We will also explore the importance of structured sparsity, with a particular focus on the 2:4 sparsity pattern. Finally, we will conduct an in-depth analysis of our proposed accelerator’s architecture, detailing its design choices.

4.1. The 2:4 Fine-Grained Structured Sparsity Pattern

Exploiting sparsity in a neural network delivers several performance benefits. First, compute throughput can be increased by skipping computation on zero-value matrix elements. Second, memory bandwidth usage can be reduced by fetching only the non-zero elements. Third, for latency-bound inferencing applications, latency can be reduced by fetching more of the nonzero values from memory and storing them locally on-chip.

4.1.1 Disadvantages of Unstructured Sparsity

While numerous approaches exist for pruning neural networks to achieve high levels of sparsity, no single method has been universally accepted for maintaining accuracy while also delivering inference speedup on hardware. Therefore, extracting real performance gains from hardware through sparsity remains an open problem, as unstructured sparsity often leads to **irregular memory accesses** and **underutilization of computational resources**.

4.1.2 Fine-Grained Structured Sparsity

NVIDIA introduces Fine-Grained Structured Sparsity, a novel approach which **doubles compute throughput** for deep neural networks. Sparsity is possible in deep learning because the importance of individual weights evolves during the learning process, and by the end of network training, only a subset of weights have acquired a meaningful purpose in determining the learned output. The remaining weights are no longer needed. Fine-grained structured sparsity imposes a constraint on the allowed sparsity pattern, making it more efficient for hardware to do the necessary alignment of input operands.

4.1.3 Two-to-Four (2:4) Sparsity Pattern

Structure is enforced through a 2:4 sparse matrix definition, a sparsity pattern that balances **hardware efficiency and model accuracy**. In this scheme, **at least 2 out of every 4 consecutive elements** in a weight matrix are pruned to **zero**. Compared to unstructured sparsity, which introduces **random sparsity patterns** that are difficult to exploit at the hardware level, **2:4 sparsity maintains a predictable structure**, enabling efficient memory access, optimized dataflows, and computational speedups.

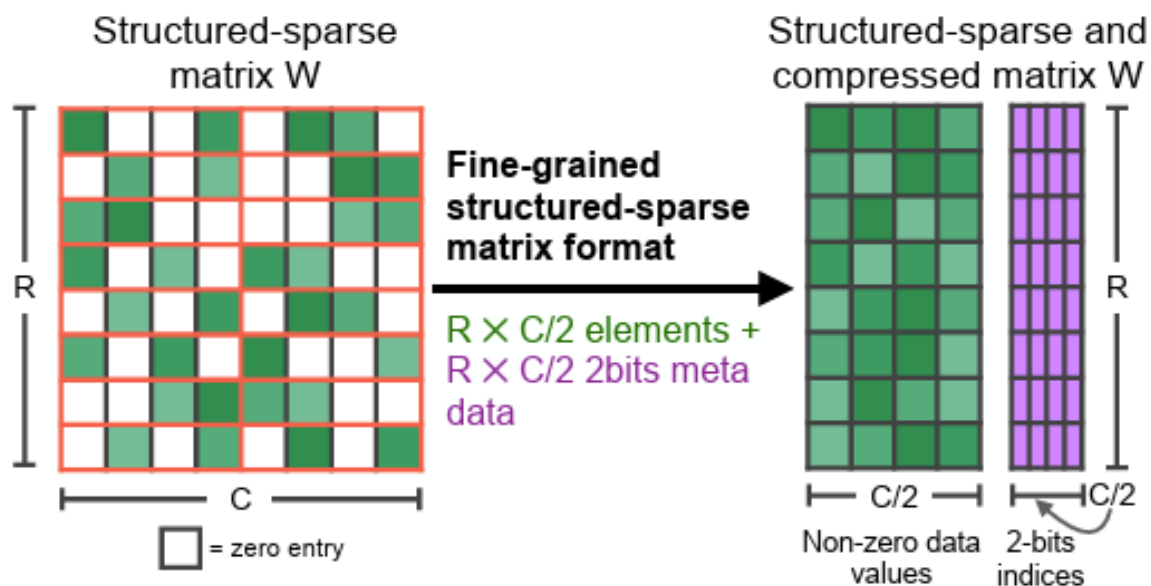


Fig.1. Structured-sparse matrix (W) storage format. The uncompressed matrix is of dimensions $R \times C$ and the compressed matrix is of dimension $R \times \frac{C}{2}$.

4.1.4 Sparse Matrix Multiply-Accumulate (MMA) Operations

Sparse MMA instructions skip the compute on entries that have zero values, resulting in a doubling of the Tensor Core compute throughput. For example, in **Fig.2.** below, Matrix A is a Sparse matrix with **50% sparsity** following the required 2:4 structured pattern and Matrix B is a dense matrix of half the size. A standard

MMA operation would not skip the zero values and would compute the result for the entire 16x8x16 matrix multiply in N cycles. Using a Sparse MMA instruction, only the elements in each row of Matrix A that have a non-zero value are matched with the corresponding elements from Matrix B. This transforms the computation into a smaller matrix multiply that takes just $N/2$ cycles, a **2x speedup**.

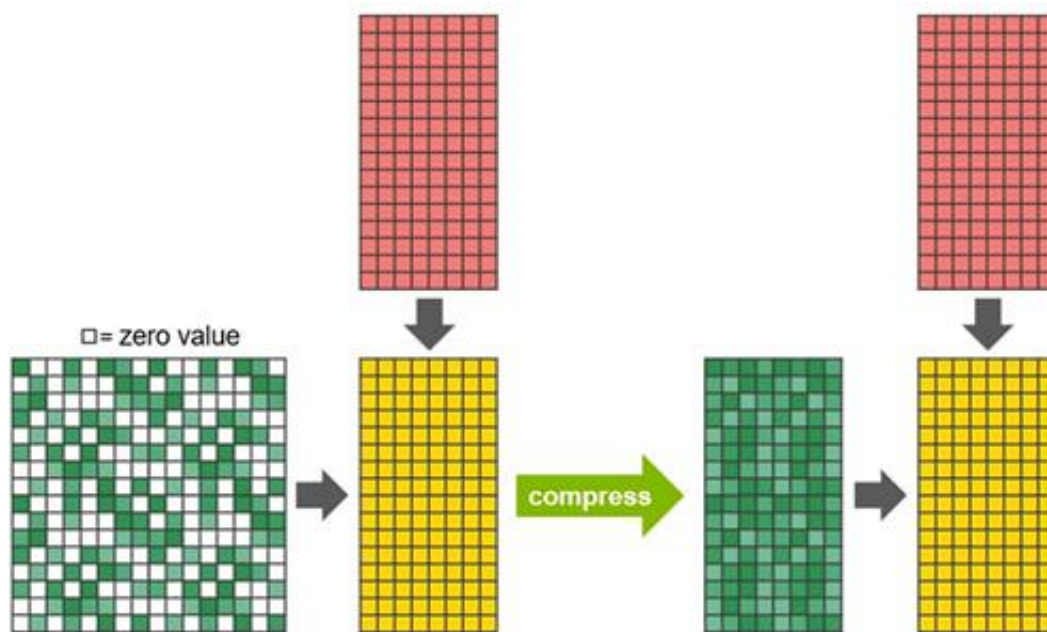


Fig.2. Dense MMA and Sparse MMA operations using 16x16 sparse matrix (Matrix A), multiplied by a dense 16x8 matrix (Matrix B). Sparse MMA operation on right doubles throughput by skipping compute of zero values.

4.1.5 Why 2:4 Sparsity?

When applied to a matrix, the 2:4 pattern has the following benefits over alternative sparsity approaches:

- **Efficient Memory Access:** Unstructured sparsity patterns lead to poor **cache utilization** and **higher latency** due to data-dependent accesses in formats like CSR/CSC/COO. In contrast, **2:4 sparsity** maintains a consistent

sparsity pattern across sub-blocks, enabling **better memory bandwidth utilization** and eliminating the need for **indirection**, as nonzero positions can be directly inferred from the compression rate directly.

- **Balanced Accuracy and Performance:** Unlike extreme sparsity patterns, which may degrade model accuracy, **2:4 sparsity has been shown to maintain inference accuracy** while achieving **significant computational speedups**.
- **Compressed Format Efficiency:** Using **CSR format** for unstructured sparsity introduces high **storage overhead** (up to **200%**) due to metadata. In contrast, the **2:4 sparse format** requires only **2-bit metadata per value**, reducing overhead to **25% for 8-bit** values. Looking at 4 dense 8-bit elements, they require $4 * 8 = 32$ -bits of storage while 2:4 sparsity for 8-bit operands $2*8 + 2*2 = 20$ bits, $\frac{32-20}{32} \cdot 100 \cong 38\%$ **savings for 8-bit operands**, significantly improving **memory capacity and bandwidth efficiency**.

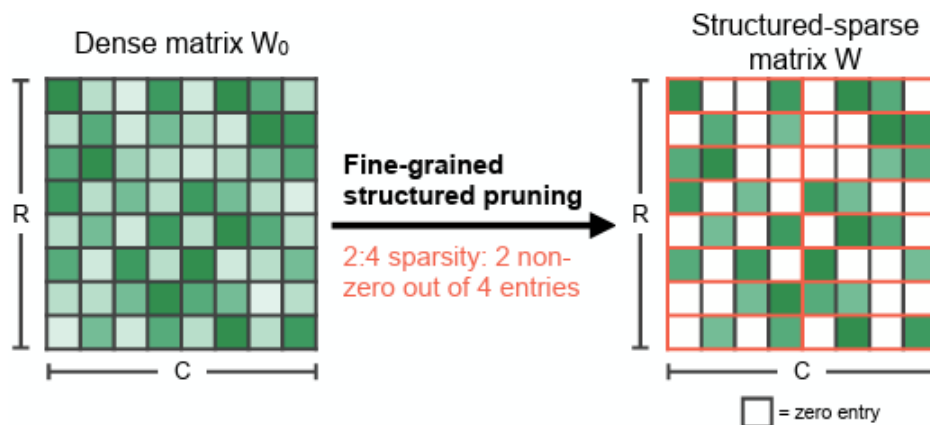


Fig.3. Step 2 of the workflow modifies the weights in the dense matrix (W_0) on the left to conform to the 2:4 constraint, yielding the sparse matrix (W) on the right. (Lighter colors signify values closer to zero).

4.1.6 Network Pruning Workflow

While the network pruning workflow is not the main focus of this work, a brief discussion is essential to validate the **accuracy and effectiveness** of the 2:4 sparsity pattern. NVIDIA has developed a simple and universal recipe for sparsifying deep neural networks for inference using this structured sparsity pattern, that **maintains original accuracy**, and **avoids any hyper-parameter searches** in a **3-step workflow**.

- **Step 1:** The network is first trained using dense weights (without sparsity).
- **Step 2:** Then fine-grained structured pruning is applied that removes the two smallest weights in each group of 4 to meet the 2:4 pattern illustrated in **Fig.3.** above.
- **Step 3:** The remaining non-zero weights are retrained to recover model accuracy lost when half of the weights are removed from **Step 2**. To avoid any hyper-parameter search, we simply repeat the training session from **Step 1**, starting with weights from **Step 1** rather than a random initialization.

This method results in virtually **no loss in inferencing accuracy** based on evaluation across dozens of networks spanning vision, object detection, segmentation, natural language modeling, and translation.

4.2. NVIDIA's Ampere Tensor Core Architecture

Tensor Cores accelerate **matrix multiply-and-accumulate (MMA) instructions**, which are essential for neural network layers that perform operations like **convolutions, linear layers, recurrent cells, and transformer blocks**. The Ampere architecture introduces significant advancements, particularly in its Tensor Core architecture, memory hierarchy, and support for structured sparsity.

These Tensor Cores support a variety of data types, including FP16, BF16, TF32, FP64, INT8, INT4, and binary, providing flexibility for different computational needs.

Each Tensor Core can perform 256 FP16/P32 fused multiply-add (FMA) operations per clock cycle, resulting in a total of 1,024 such operations per clock cycle. This represents a 2x increase in computational power. The Tensor Cores are designed to

exploit fine-grained structured sparsity and thus effectively doubling the performance of standard Tensor Core operations by skipping the computation of zero-valued weights.

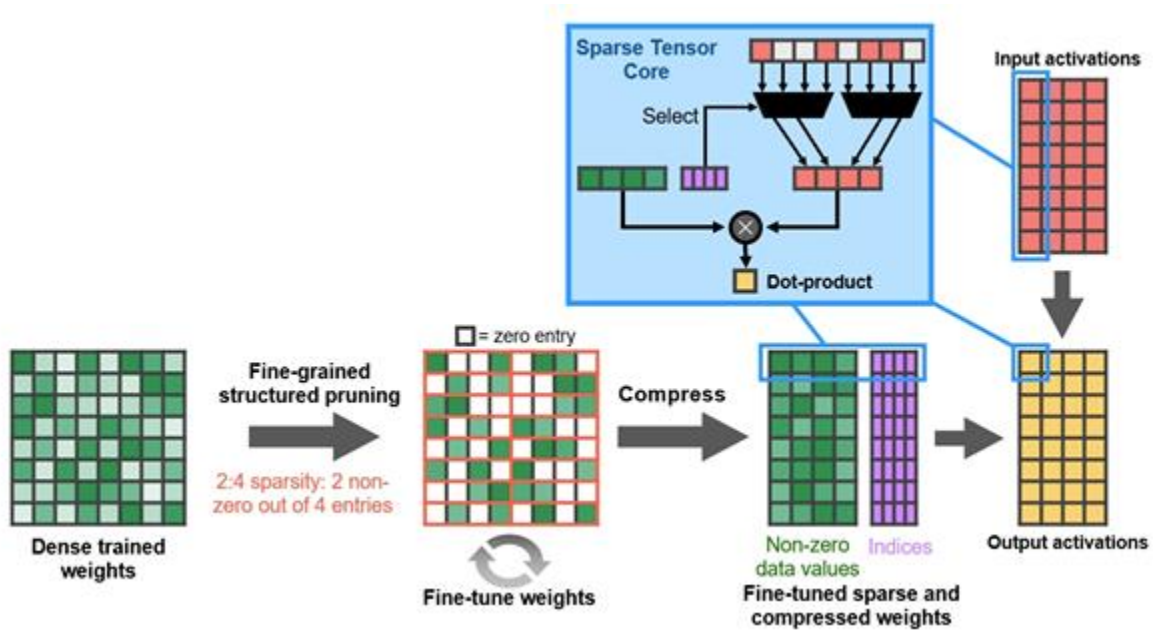


Fig.4. Structured Sparsity prunes trained weights using a 2-out-of-4 non-zero pattern, followed by a straightforward and universal fine-tuning process for the remaining weights. This approach compresses weights, achieving a 2× reduction in both data footprint and bandwidth. Additionally, it provides a look into the sparse Tensor Core architecture.

4.2.1 Weight Selection and Data Alignment

As shown in **Fig.4.** the compressed sparse matrix is fed into the Tensor Core, which is optimized for 2:4 sparsity. The Tensor core efficiently selects the nonzero weight elements using the metadata matrix. The selected weights and their corresponding input activations are aligned in hardware for efficient dot-product computation. This alignment ensures that **all Tensor Core execution units are fully utilized**, avoiding wasted cycles caused by unnecessary zero multiplications.

4.2.2 Dot-Product Computation on Tensor Cores

Each **Tensor Core** performs a **matrix multiply-accumulate (MMA)** operation using only the selected **nonzero weights** by two **4-to-2 multiplexers** where the indices serve as the select values. The MMA operation is done on a **4x1 dense activation block** (after weighted selection), multiplied with **1x4 sparse weight block** using **4 multipliers** and **3 accumulators**. The results are then added to the corresponding accumulator element using an additional accumulator before being stored back to the accumulator matrix (output activations).

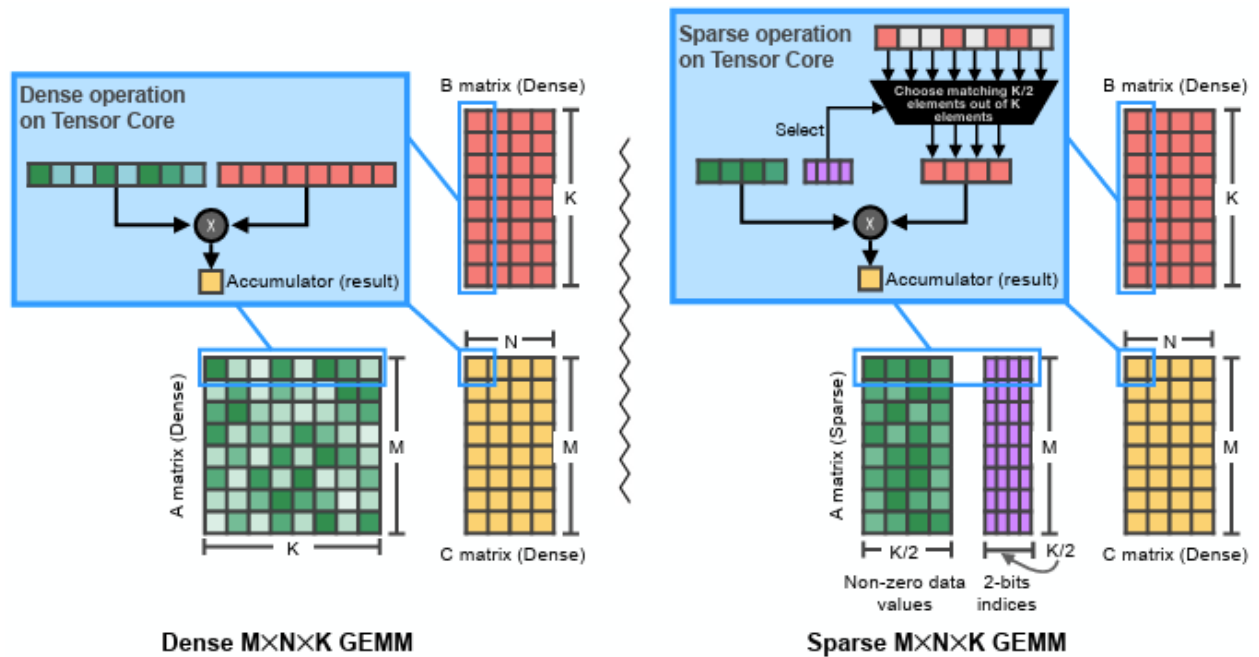


Fig.5. The different structure of Dense operations on a Tensor Core and Sparse operations on a Tensor Core.

4.2.3 Dense GEMM operations VS Sparse GEMM operations

In **Fig.5.** we compare the execution process of a Dense operation versus a Sparse operation on Tensor Cores. The Dense operation involves Matrix A an MxK dense matrix with no sparsity, where each row of the matrix is multiplied by the corresponding column of dense Matrix B (KxN dimensions). This results in K x N

multiplications and $K \times N$ additions per matrix A row, leading to a total of **$2 \times K \times N \times M$ operations** for the entire matrix multiplication.

The Sparse operation consists of sparse Matrix A(that has gone through pruning and compression to fit the 2:4 structure) of $M \times \frac{K}{2}$ dimensions followed by $M \times \frac{K}{2}$ metadata matrix and a dense Matrix B ($K \times N$ dimensions). This results in $\frac{K}{2} \times N$ multiplications and $\frac{K}{2} \times N$ additions per matrix A row. For the whole matrix multiplication, we have a total of $2 \times \frac{K}{2} \times N \times M = K \times N \times M$ **operations**. That is **half of the number of operations used in the dense GEMM**.

Input Operands	Accumulator	Dense TOPS	Sparse TOPS
FP32	FP32	19.5	-
TF32	FP32	156	312
FP16	FP32	312	624
BF16	FP32	312	624
FP16	FP16	312	624
INT8	INT32	624	1248

Table 1. Tensor Core input / output formats (numeric precision) and performance in tera-operations per second (TOPS)

Sparse Tensor Cores support FP16, BF16, and 8b-integer input / output types. Furthermore, TF32 mode is supported for FP32 input / output but the pattern becomes 1:2 sparse. Peak dense and sparse Tensor Core throughputs are shown in **Table 1**.

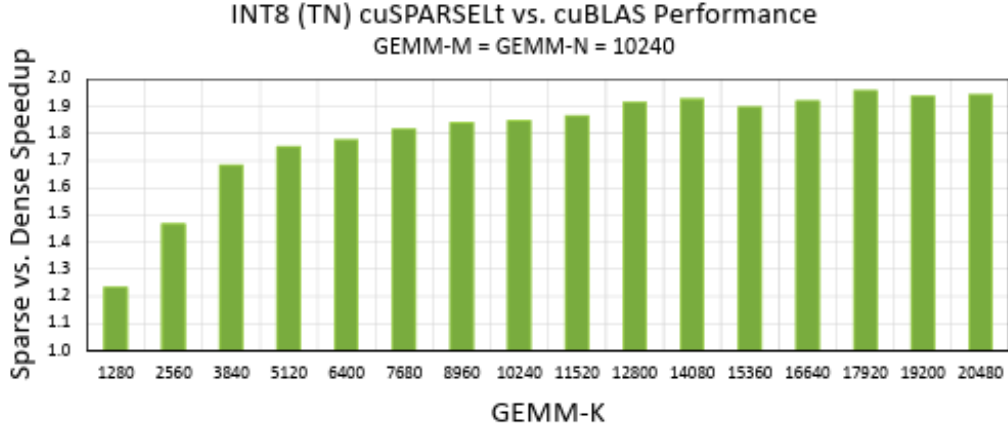


Fig.6. Comparison of sparse and dense INT8 GEMMs on Ampere Tensor Cores. Larger GEMMs achieve nearly a 2x speedup with Sparse Tensor Cores.

Speedups for GEMM dimensions are shown, with larger GEMMs that achieve higher arithmetic intensity, they approach the 2x speedup of Sparse Tensor Cores.

4.3 Our Sparse Matrix Multiplication Accelerator Architecture

Our Sparse Tensor Core, inspired by the NVIDIA Ampere architecture, is optimized for the 2:4 sparsity pattern (as described in Section 4.1) and supports **signed INT8** data types. Currently implemented on the **CVA6 (formerly Ariane)** processor via the **CVXIF interface**, the accelerator operates directly on the CPU's register file due to the current lack of a memory interface through CVXIF. The target hardware is the cv64a6_imafdc_sv39, a 64-bit version of CVA6 that includes the IMAFDC extensions.

This Sparse Tensor Unit performs Matrix-Matrix Accumulation (MMA) operations between:

1. A **sparse 4x2 matrix (A)** containing signed **INT8** elements, accompanied by a corresponding **4x2 metadata matrix** with **2-bit** entries. Each metadata element maps directly to an element in the sparse matrix.
2. A **dense 4x4 matrix (B)** containing **signed INT8** elements.
3. The results of the MMA operation are accumulated in a **4x4 dense matrix (C / Accumulator)** of **signed INT32** elements.

See Fig.7.

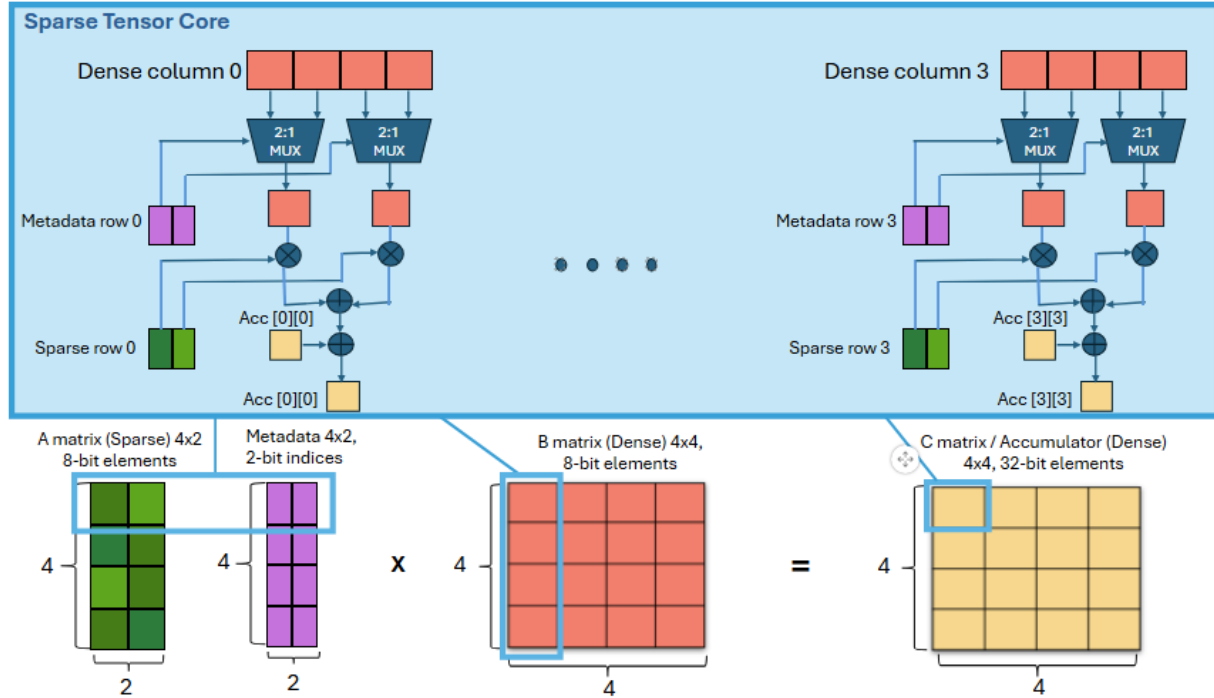


Fig.7. The Sparse Tensor Core utilizes 16 parallel Sparse Matrix Multiply-Accumulate (MMA) units, inspired by the Ampere Tensor Core, to concurrently compute the elements of the accumulator matrix (C).

4.3.1 Sparse Tensor Core Architecture

The proposed architecture is illustrated in **Fig.7**. To maximize efficiency, all necessary arrays are stored on-chip in dedicated memory elements, maintaining the same dimensions and data sizes. This allows the accelerator to **immediately begin computations** without requiring a complex pipeline structure.

The Tensor Core features **16 parallel Matrix-Multiply-Accumulate (MMA) units**, inspired by the Ampere Architecture. These units execute all required matrix operations in parallel, delivering results **within a single clock cycle**.

The use of compact INT8 matrix elements not only reduces memory and bandwidth demands but also **optimizes power efficiency**. This compact design allows for **additional hardware resources** to be incorporated without significantly

increasing power consumption, making the accelerator both **highly efficient and scalable**.

In **Section 5: Implementation**, we will explore how the Accelerator loads and stores data to and from the register file, detailing its workflow and memory access patterns. Additionally, we will showcase the custom instructions designed to communicate with the accelerator.

4.3.2 Benefits of INT8 data type and Compact Matrices

INT8 (Signed 8-bit integer): This data type provides a **balance in precision and storage efficiency**. It is sufficient for many machine learning inference tasks while simultaneously offering hardware efficiency due to the fact that arithmetic units for INT8 are generally smaller and faster than those for larger data types. Additionally multiplications with smaller data types require **less power and fewer cycles** than floating-point operations.

Compact Matrixes: Smaller matrix dimensions allow for a **reduction in memory bandwidth** due to less data being fetched needing less storage in the on-chip memory of the accelerator. Another benefit is the **simplified logic** often followed with smaller matrix sizes leading to **faster operations and lower power consumption**. Smaller matrixes are also easier to **process in parallel** as shown in the demonstrated architecture above in **Fig.7**. with the utilization of 16 parallel MMA units in order to finish the matrix multiplication in 1 cycle while also providing greater flexibility that can be useful to the software. Lastly looking at the metadata matrix it is essential to be as compact as possible so that it doesn't negate the memory savings gained by sparsity and compression while preserving adequate structural information.

5. Implementation

The **Sparse Matrix Accelerator** was integrated into the **CVA6 processor** using a set of **custom RISC-V instructions**, enabling efficient communication between the

processor and the accelerator. These instructions handle matrix loading, computation initiation, and result storage.

To validate the design, a **software emulation in C** was developed to replicate the accelerator’s functionality. This allowed for **early-stage verification**, ensuring correct results by comparing them against a **baseline software implementation** running on CVA6. The emulation facilitated debugging and design refinement before hardware synthesis, reducing development time.

The Register-Transfer Level (RTL) design of the accelerator was implemented in SystemVerilog, incorporating parallel Matrix-Multiply-Accumulate (MMA) units.

5.1 Custom Instruction Set Extensions

The Accelerator supports **13 different R-Type** custom instructions implemented through CVXIF.

32-bit RISC-V R-Type instructions follow the format bellow in **Fig.8**.

Instruction Formats	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Register/register	funct7							rs2				rs1				funct3				rd				opcode								

Fig.8. 32-bit RISC-V R-Type instruction formats

Custom Instructions	Funct7	funct3	Opcode
RESET_ACC	0000000	000	0001011
LOAD_DENSE	0000000	001	0001011
LOAD_DENSE_START	0000001	001	0001011
LOAD_SPARSE	0000000	010	0001011
LOAD_SPARSE_START	0000001	010	0001011
STORE_ACC0	0000000	011	0001011

STORE_ACC2	0000010	011	0001011
STORE_ACC4	0000100	011	0001011
STORE_ACC6	0000110	011	0001011
STORE_ACC8	0001000	011	0001011
STORE_ACC10	0001010	011	0001011
STORE_ACC12	0001100	011	0001011
STORE_ACC14	0001110	011	0001011

Table.2. Accelerator Custom Instructions Opcode, funct3, funct7.

Using one of RISC-V's custom opcodes we create **13 different custom instruction**.

Within the custom instructions the same opcode (0001011) is maintained. Different types of instructions are distinguished through funct3(000, 001, 010, 011). Any distinctions within the same type of instruction (same funct3) are done through funct7.

RESET_ACC: is used to reset the 4x4 32-bit accumulator inside the accelerator to zero.

- **LOAD_DENSE:** This instruction **loads the 4x4 8-bit dense matrix into the accelerator's on-chip memory**. The first two rows (8-bit $\times$ 8 elements = 64 bits) are stored in register rs1, with row 0 occupying the lower 32 bits (LSBs) and row 1 in the upper 32 bits (MSBs). Similarly, the third and fourth rows are stored in register rs2, where row 2 is placed in the lower 32 bits and row 3 in the upper 32 bits.
- **LOAD_DENSE_START:** This instruction functions identically to **LOAD_DENSE**, with the added capability of **initiating the multiplication operations** within the **Sparse Tensor Core** between the **sparse and dense matrices**. It is the **software's responsibility** to ensure that the

sparse matrix is already loaded into the accelerator's memory before invoking this instruction.

- **LOAD_SPARSE:** This instruction **loads the 4x2 8-bit sparse matrix and the 4x2 2-bit metadata matrix into the accelerator's on-chip memory**. The whole sparse matrix fits into rs1 since 8-bit x 8-elements = 64bits as well as the metadata matrix into rs2.
- **LOAD_SPARSE_START:** This instruction functions identically to **LOAD_SPARSE**, with the added capability of **initiating the multiplication operations** within the **Sparse Tensor Core** between the **sparse and dense matrices**. It is the **software's responsibility** to ensure that the **dense matrix is already loaded** into the accelerator's memory before invoking this instruction.
- **STORE_ACCX:** The STORE_ACC instructions are responsible for returning the results of the Matrix-Multiplication that are located on the accelerators accumulator back to the CPU's register file. Because the accumulator contains 32-bit values we can only load 2 elements(64bits) at a time into rd to return back to the CPU. That why we need $16 / 2 = 8$ different instructions where funct7 indicated which two sequential elements are returned. For example, STORE_ACC0 returns elements 0 and 1 (ACC[0][0] and ACC[0][1]), STORE_ACC2 returns elements 2 and 3 (ACC[0][2] and ACC[0][3]), STORE_ACC4 returns elements 4 and 5 (ACC[1][0] and ACC[1][1]) and so on.

5.1.1 Custom Instruction Integration on CVXIF

CVA6 includes a `cvxif_instr_pkg` package, which contains multiple structs, such as `opcode_t` and other structures used in CVIF request and response handling. This package served as a template for integrating our own instructions. Each instruction is stored in the `CoproInstr` array, which organizes them in the following format:

```

parameter int unsigned NbInstr = 13;
parameter copro_issue_resp_t CoproInstr[NbInstr] = '{
    '{
        // Custom RESET_ACC
        instr:
        32'b00000_00_00000_00000_0_00_00000_0001011,
        mask: 32'b11111_11_00000_00000_1_11_00000_1111111,
        resp : '{accept : 1'b1, writeback : 1'b0, register_read : {1'b0, 1'b0, 1'b0}},
        opcode : RESET_ACC
    },

```

- **instr:** Stores the binary representation of each instruction
- **mask:** Has 1s in the positions where we want to preserve the value of instr and 0s where we know those bits are going to change like the bits for rs1, rs2 and rd.
- **resp:** Contains information used in CVXIF. Accept determined whether this instruction is accepted by the accelerator. Writeback determines if this instruction writes back to the CPU's register file or not. Register_read is a 3-bit value that determines what source registers are read by this instruction {rs3 (if supported), rs2, rs1}.

5.2 Sparse Matrix Accelerator Emulation

In order to start the hardware implementation of the Accelerator on CVA6, we first created a Software emulation of the Tensor Core in C.

This helped better understand:

5.2.1 Base Blocked Matrix Multiplication Algorithm

Blocked matrix multiplication is a technique for multiplying large matrices by breaking them down into smaller blocks (or sub-matrices) and processing those sub-blocks individually to calculate the final multiplication matrix. This allows us to use our Accelerator that only processes static dimension arrays for every sub-block in order to complete the whole matrix-multiplication on the larger matrices. Since our Accelerator contains 4x4 size matrices the block size is 4.

```

void matrix_mult(int B, int M, int K, int N, int8_t sparse[M][K], int8_t dense[K][N]) {
    int32_t result;

    /*for (int a = 0; a < M; a++) { ...

    // Perform blocked multiplication row by row for blocks in row-major order.
    for (int ii = 0; ii < M; ii += B) {          // Block rows of Y and X
        for (int jj = 0; jj < N; jj += B) {      // Block columns of Z and X
            for (int kk = 0; kk < K; kk += B) {  // Block columns of Y and rows of Z
                for (int i = ii; i < min(ii + B, M); i++) { // Rows in the current block
                    for (int j = jj; j < min(jj + B, N); j++) { // Columns in the current block
                        result = 0;
                        for (int k = kk; k < min(kk + B, K); k++) { // Overlapping blocks
                            result += (int32_t) sparse[i][k] * dense[k][j]; // Dot product for the block
                        }
                        final_matrix[i][j] += result;
                    }
                }
            }
        }
    }
}

```

Fig.9. Blocked Matrix Multiplication Algorithm without the Accelerator

In **Fig.9.** We can see the standard Matrix Multiplication Algorithm without the integrations of the Accelerator. In matrix multiplication M = number sparse rows, N = number of dense columns, K = number of sparse columns and dense rows, B = block size.

With this information we can understand that the algorithm starts by selecting sparse blocks of the same rows and dense blocks of the same columns. Then it moves using the same rows sparse blocks but with dense blocks of different columns until all the dense columns blocks are used in the algorithm and lastly, we start the process all over again for sparse blocks of different rows.

We can see this by the order of the 3 loops. The top loop remains the same while the second loop also remains the same and the third loop that determines sparse columns and dense rows changes. After all the blocks that have the same sparse rows and same dense columns are complete the second loop progresses to blocks of different dense columns but the outer loop maintains sparse blocks of the same rows and lastly the rows of the sparse blocks change due to the outer loop restarting the same process. These steps can change just by changing the order of the different loops.

For every chosen sub-block the 3 inner-most loops perform the matrix multiplication between the sparse sub-block and the corresponding dense sub-block using standard matrix multiplication.

Blocked Matrix Multiplication provides additional **benefits** to the standard Matrix Multiplication. Smaller blocks are more likely to fit in the cache, improving cache locality and thus, **reducing cache misses and speeding up computation**. Block multiplications can be done **independently**, making it easy to **parallelize the process** and lastly it allows for the handling of matrices larger than the available RAM since blocks can be loaded and processed from memory one at a time.

5.2.2 Custom Instruction Placement in Algorithm

From the Blocked Matrix Multiplication algorithm, we progressed to determining where our custom accelerator instructions should be placed to efficiently perform matrix multiplication. As observed in **Fig. 9.**, the three innermost loops handle element selection and multiplication within the sub-blocks defined by the outer loops.

Since our accelerator inherently performs blocked multiplication, these inner loops become redundant when integrating our accelerator. Instead of executing these loops, we replace them with custom instructions that load the dense and sparse matrix sub-blocks (that are determined by the outer loops) into the accelerator.

Additionally, we must ensure that blocks corresponding to the same **K** index are accumulated correctly to produce the final output block. To achieve this, we reset the accumulator after all sub-blocks with the same **K** are processed. This means that, after the third loop (inside the second loop), we store the accumulator's contents into our emulated register file and then reset the accumulator before proceeding.

5.2.3 Software Emulation of the Sparse Matrix Accelerator

The software emulation of our accelerator's design played a crucial role in understanding the internal structure of the tensor core. It helped identify the necessary hardware components and how they are going to communicate with each other, determine how the processor would handle custom instructions, and define the software workflow required to interface with the accelerator effectively.

Additionally, this emulation significantly streamlined the process of writing the hardware code, making **development more efficient** and **reducing potential design errors**.

The software emulation includes array-based representations of the hardware memory components and employs a set of opcodes to distinguish between the various custom instructions implemented in the emulated tensor core.**Fig.10.**

```
#define OPCODE 11      //opcode: 0001011
#define LOAD_DENSE 1   //funct3: 001
#define LOAD_SPARSE 2 //funct3: 010
#define RESET_ACC 0    //funct3: 000
#define STORE_ACC 3    //funct3: 011
#define min(x, y) ((x) < (y) ? (x) : (y))

int64_t REG_FILE[32]; //Memory elements accessed from both the accelerator and the program
int8_t sparse[4][2], meta_data[4][2], dense[4][4];
int32_t ACC[4][4];
```

Fig.10. Emulated hardware memory elements and custom instruction opcode definitions

The Tensor Core is a function that takes an `int32_t instr` argument, and from that instruction extracts the opcode, rd, funct3, rs1, rs2, funct7 (In the hardware implementation CVXIF in collaboration with a custom instruction decoder provides us with those parameters). Based on the opcode, the system calls different functions with the arguments extracted before to execute various instructions, such as loading dense and sparse matrices (`load_dense_matrix`, `load_sparse_matrix`), resetting the accumulator (`reset_acc`), and storing accumulator values (`store_acc`).

Furthermore, the Software emulation of the Accelerator was used to **verify the results and behavior** of the actual Tensor Core.

5.3 Sparse Matrix Accelerator Implementation

This section analyzes the hardware implementation of the Sparse Matrix Accelerator as well as the Software that uses it to perform matrix multiplication operations.

5.3.1 Sparse Matrix Generator

To test the **Sparse Tensor Core**, rather than relying on a tensor library like **TensorFlow**, we developed **sparse_matrix_generator.py**. This script generates **randomized sparse, metadata, and dense matrices** based on user-defined parameters:

- **M** (number of sparse rows)
- **K** (number of sparse columns & dense rows)
- **N** (number of dense columns)

The generated matrices contain **random signed INT8 values** ranging from **-128 to 127**. The **sparse matrix** follows a **2:4 sparsity pattern** in its **uncompressed form**, with non-zero elements placed at **random positions** while adhering to the **2:4 constraint**. Their **indices** are stored in the **metadata matrix**.

The script outputs these matrices as **static arrays** in a **datasetX.h** file, which is then used as input for the **C program** that uses the accelerator.

5.3.2 Hardware Implementation

The implemented architecture includes a **custom instruction decoder** and the **Tensor Core**.

Leveraging the CVXIF interface on CVA6, the decoder fetches, decodes, and dispatches instructions to the Tensor Core.

A coprocessor alu template was provided by CVA6. Our design keeps the input/output signals, the parameters and the logical elements needed for CVXIF as they are with the addition of the input signal **rs_valid_i** (as shown bellow) that is needed to verify the validity of both source registers when trying to start the matrix multiplication in order to obtain correct results.

```
import cvxif_instr_pkg::*;
#(
    parameter int unsigned NrRgprPorts = 2,
    parameter type readregflags_t = logic,
    parameter type hartid_t = logic,
    parameter type id_t = logic,
    parameter type registers_t = logic
) (
    input logic clk_i,
    input logic rst_ni,
    input registers_t registers_i,
    input readregflags_t rs_valid_i,
    input opcode_t opcode_i,
```


There are added memory elements to house the matrices needed during the multiplication process:

```
logic [63:0] result_n, result_q;
hartid_t hartid_n, hartid_q;
id_t id_n, id_q;
logic valid_n, valid_q;
logic [4:0] rd_n, rd_q;
logic we_n, we_q;
logic signed [7:0] sparse[3:0][1:0];
logic signed [7:0] sparse_n[3:0][1:0];
logic [1:0] meta_data[3:0][1:0];
logic [1:0] meta_data_n[3:0][1:0];
logic signed [7:0] dense[3:0][3:0];
logic signed [7:0] dense_n[3:0][3:0];
logic signed [31:0] ACC[3:0][3:0];
logic signed [31:0] ACC_n[3:0][3:0];
```

The **combinatorial logic** inside the tensor core assigns the previous cycle register's values as input for the next cycle in order to maintain the matrices values throughout the execution, giving the option for **optimizations and reusability** without the need to reload matrices every time. It assigns default values to the cvxif signals and contains a case statement that depending on the **opcode** changes the state of the accelerator.

- If a **RESET_ACC** is fetched simply set to zero ACC_n:

```
case (opcode_i)
  cvxif_instr_pkg::RESET_ACC: begin
    ACC_n[0] = {32'b0, 32'b0, 32'b0, 32'b0};
    ACC_n[1] = {32'b0, 32'b0, 32'b0, 32'b0};
    ACC_n[2] = {32'b0, 32'b0, 32'b0, 32'b0};
    ACC_n[3] = {32'b0, 32'b0, 32'b0, 32'b0};
  end
```

- If a **LOAD_DENSE** instruction is fetched we extract from rs1 and rs2 the 8-bit values of the dense matrix and store them in dense_n.

```

dense_n[0][0] = registers_i[0][7:0];
dense_n[0][1] = registers_i[0][15:8];
dense_n[0][2] = registers_i[0][23:16];
dense_n[0][3] = registers_i[0][31:24];
dense_n[1][0] = registers_i[0][39:32];
dense_n[1][1] = registers_i[0][47:40];
dense_n[1][2] = registers_i[0][55:48];
dense_n[1][3] = registers_i[0][63:56];
dense_n[2][0] = registers_i[1][7:0];
dense_n[2][1] = registers_i[1][15:8];
dense_n[2][2] = registers_i[1][23:16];
dense_n[2][3] = registers_i[1][31:24];
dense_n[3][0] = registers_i[1][39:32];
dense_n[3][1] = registers_i[1][47:40];
dense_n[3][2] = registers_i[1][55:48];
dense_n[3][3] = registers_i[1][63:56];

```

- If a **LOAD_DENSE_START** is fetched, data from registers rs1 and rs2 are loaded into dense_n as shown above, and the matrix multiplication process begins when both registers hold valid data. Sixteen parallel multiply-accumulate (MAC) units then perform the core computation as described in **section 4.3**. Each unit processes a row of the sparse matrix, multiplying each non-zero element with the corresponding element from dense_n (as indicated by the metadata), and accumulating the results. The outputs of the MAC units combine to form the accumulator matrix.

```

if (rs_valid_i == 2'b11) begin //make sure both registers are valid so that if there are invalid values in the registers that transfer dense_n, the multiplication is not executed
  for (int i = 0; i < 4; i++) begin
    ACC_n[i][0] = sparse_n[i][0]*dense_n[meta_data_n[i][0]][0] + sparse_n[i][1]*dense_n[meta_data_n[i][1]][0] + ACC_n[i][0];
    ACC_n[i][1] = sparse_n[i][0]*dense_n[meta_data_n[i][0]][1] + sparse_n[i][1]*dense_n[meta_data_n[i][1]][1] + ACC_n[i][1];
    ACC_n[i][2] = sparse_n[i][0]*dense_n[meta_data_n[i][0]][2] + sparse_n[i][1]*dense_n[meta_data_n[i][1]][2] + ACC_n[i][2];
    ACC_n[i][3] = sparse_n[i][0]*dense_n[meta_data_n[i][0]][3] + sparse_n[i][1]*dense_n[meta_data_n[i][1]][3] + ACC_n[i][3];
  end
end
end

```

- If a **LOAD_SPARSE** instruction is fetched we load the data from rs1 to sparse_n and data from rs2 to meta_data_n.

```

sparse_n[0][0] = registers_i[0][7:0];
sparse_n[0][1] = registers_i[0][15:8];
sparse_n[1][0] = registers_i[0][23:16];
sparse_n[1][1] = registers_i[0][31:24];
sparse_n[2][0] = registers_i[0][39:32];
sparse_n[2][1] = registers_i[0][47:40];
sparse_n[3][0] = registers_i[0][55:48];
sparse_n[3][1] = registers_i[0][63:56];

meta_data_n[0][0] = registers_i[1][1:0];
meta_data_n[0][1] = registers_i[1][9:8];
meta_data_n[1][0] = registers_i[1][17:16];
meta_data_n[1][1] = registers_i[1][25:24];
meta_data_n[2][0] = registers_i[1][33:32];
meta_data_n[2][1] = registers_i[1][41:40];
meta_data_n[3][0] = registers_i[1][49:48];
meta_data_n[3][1] = registers_i[1][57:56];

```

- If a **LOAD_SPARSE** instruction is fetched we load the data from rs1 to sparse_n and data from rs2 to meta_data_n as above and start multiplication using exact same for loop that creates the 16 MMA units like previously.
- If any **STORE_ACCX** is fetched then return to rd the two Accumulator elements indicated by **X** and set we_n = 1'b1 so that cvxif knows to enable register write back to CPUs register file. In the figure bellow we showcase STORE_ACC0 and STORE_ACC2 but the implementation for the others is the same.

```

cvxif_instr_pkg::STORE_ACC0: begin
    result_n = {ACC[0][0],ACC[0][1]};
    we_n      = 1'b1;
end
cvxif_instr_pkg::STORE_ACC2: begin
    result_n = {ACC[0][2],ACC[0][3]};
    we_n      = 1'b1;
end

```

The **sequential logic** at posedge assigns the **combinatorial logic** to the registers and in case of rst negedge all the registers are reset to zero.

```

always_ff @(posedge clk_i, negedge rst_ni) begin
  if (~rst_ni) begin
    result_q <= '0;
    hartid_q <= '0;
    id_q <= '0;
    valid_q <= '0;
    rd_q <= '0;
    we_q <= '0;
    ACC[0] <= {32'b0, 32'b0, 32'b0, 32'b0};
    ACC[1] <= {32'b0, 32'b0, 32'b0, 32'b0};
    ACC[2] <= {32'b0, 32'b0, 32'b0, 32'b0};
    ACC[3] <= {32'b0, 32'b0, 32'b0, 32'b0};
    dense[0] <= {8'b0, 8'b0, 8'b0, 8'b0};
    dense[1] <= {8'b0, 8'b0, 8'b0, 8'b0};
    dense[2] <= {8'b0, 8'b0, 8'b0, 8'b0};
    dense[3] <= {8'b0, 8'b0, 8'b0, 8'b0};
    sparse[0] <= {8'b0, 8'b0};
    sparse[1] <= {8'b0, 8'b0};
    sparse[2] <= {8'b0, 8'b0};
    sparse[3] <= {8'b0, 8'b0};
    meta_data[0] <= {2'b0, 2'b0};
    meta_data[1] <= {2'b0, 2'b0};
    meta_data[2] <= {2'b0, 2'b0};
    meta_data[3] <= {2'b0, 2'b0};
  end else begin
    result_q <= result_n;
    hartid_q <= hartid_n;
    id_q <= id_n;
    valid_q <= valid_n;
    rd_q <= rd_n;
    we_q <= we_n;
    ACC <= ACC_n;
    dense <= dense_n;
    sparse <= sparse_n;
    meta_data <= meta_data_n;
  end
end

```

6. Evaluation

The evaluation process for the Sparse Matrix Accelerator consists of two parts. Verification and Performance Analysis.

6.1 Verification

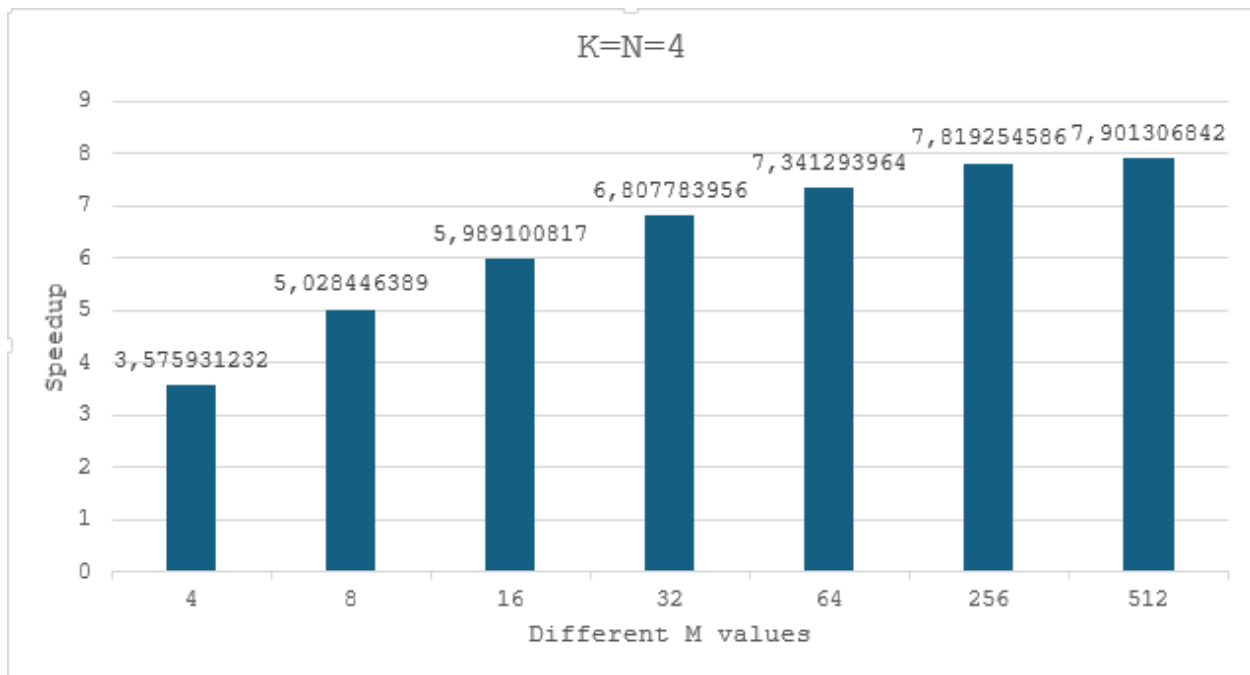
To verify our design and output results we used the Sparse Matrix Emulation algorithm see **Section 5.2.3.** in addition to the base blocked GEMM algorithm see **Section 5.2.1.**

6.2 Performance Analysis

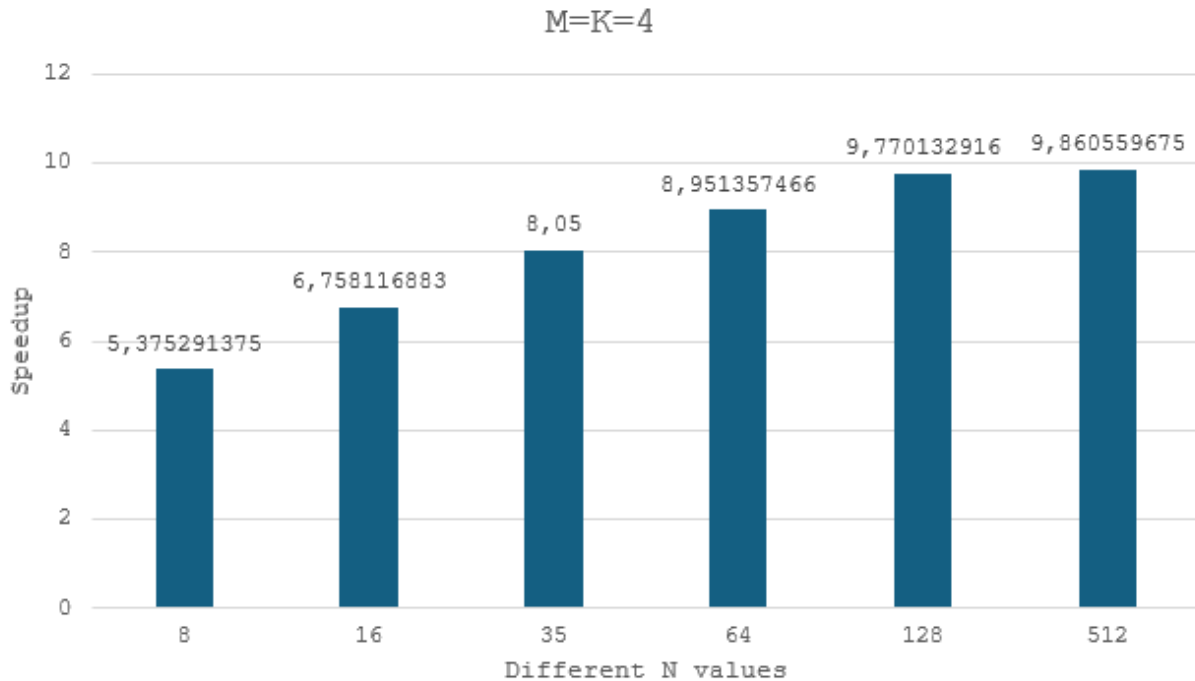
The number of **clock cycles** and **instructions** was measured for the base GEMM algorithm and the multiplication algorithm that used the Sparse Matrix Accelerator by reading the mcycle and minstret csr registers before and after each algorithm individually and subtracting to find the difference. With those measurements we are able to calculate **Speedup** = $\frac{\text{Base GEMM Cycles}}{\text{Accelerated Cycles}}$. To test our Accelerator, we measure speedup by varying M, K, N (matrix dimensions) values.

In order to obtain a comprehensive performance profile of the Accelerator a strategic set of different M, K, N values was used to obtain as much information as possible.

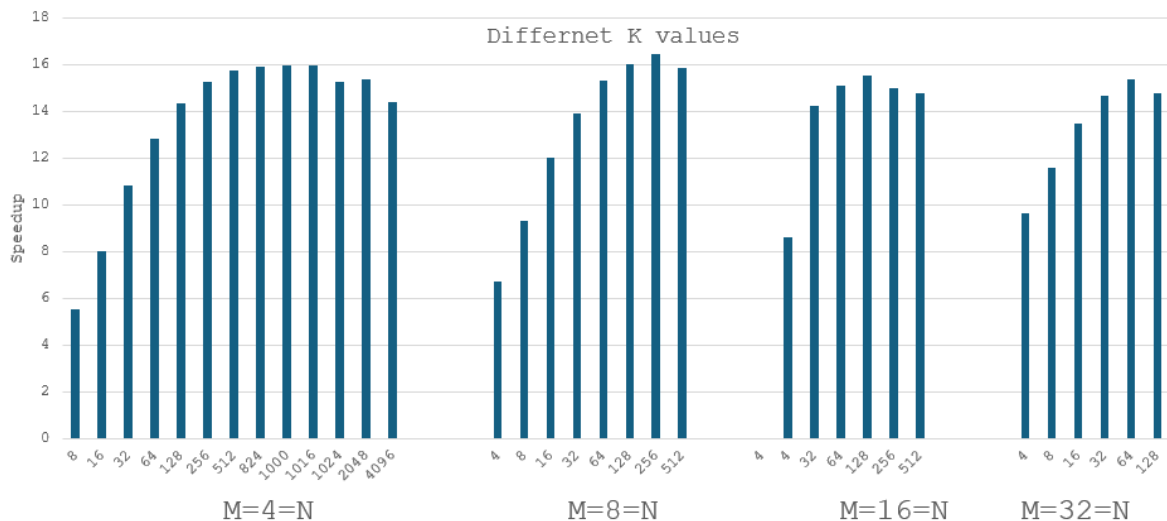
1. We started by keeping the values of K and N constant and changing only M.



2. Next, we kept the values of M and K constant and changed N.



3. Then we kept the values of M and N constant and changed only K . We observed that in comparison to the previous cases 1 and 2 when setting $M=4=N$ and changing the values of K , there was slightly higher speedup. Due to this, we run more tests for this case, setting $M=8=N$, $M=16=N$ and $M=32=N$ to see if we could maximize speedup.



4. We also tested random N , M , K values but didn't observe speedup larger than 16.4.

The sparse matrix accelerator demonstrates **increasing speedup as matrix size grows**, up to a threshold where the gains begin to taper off. Among the tested cases, increasing **M** (sparse rows) yields the **least speedup**, while increasing **N** (dense columns) results in **moderate speedup**. In contrast, increasing **K** (sparse columns, dense rows) provides the **highest speedup**, achieving **up to 16.4x faster** execution compared to the baseline algorithm. This indicates that the **accelerator scales most efficiently with K**, while scaling moderately with M and N. The strong performance with K suggests that the accelerator effectively utilizes inner-loop optimizations and parallelism, which aligns with expectations since the innermost loop is where it takes over the matrix multiplication workload.

7. Conclusions and Future Work

One of the major challenges in modern hardware design is **keeping up with the rapid growth in computational requirements** while **maintaining efficiency in terms of power, latency, and memory bandwidth**. **Sparsity in neural networks** has emerged as a critical technique to improve computational efficiency.

This work presented a sparse matrix accelerator inspired by NVIDIA's A100 Tensor Core architecture, integrated into CVA6 via the CVXIF interface. Our design focused on efficiently handling **2:4 structured sparsity**, which provides a **balance between accuracy and computational performance** while allowing for significant speedup in matrix multiplications.

While our accelerator demonstrated **significant speedup** in sparse matrix computations, **many more optimizations** can be implemented to further enhance **performance and efficiency**. One key area for improvement lies in **memory efficiency**. Optimizing how data is stored, accessed, and reused to **reduce memory bandwidth** and **minimize data movement overhead**.

Additionally, this work aligns with an **active area of research in the RISC-V ecosystem**, particularly regarding **vector extensions**. As RISC-V evolves, **vector processing units (VPUs) and specialized extensions for sparse computations** are being explored to further optimize deep learning workloads.